FIAP · 15SOAT · FASE 2

Tech-Challenge-Oficina

Sistema Integrado de Atendimento e Execução de Serviços para oficina mecânica

GRUPO	55
PARTICIPANTE	Bruno da Silva Saran
DISCORD	saranbruno
REPOSITÓRIO	github.com/saranbruno/Tech-Challenge-Oficina

SISTEMA E FUNCIONALIDADES

01

Evolução da API da oficina

A Fase 2 amplia o sistema Laravel e PostgreSQL existente com APIs para consulta, aprovação e recusa de orçamentos, comunicação com clientes e operação em Kubernetes.

Ordem de Serviço

A abertura utiliza cadastros existentes, calcula o orçamento no servidor e retorna a identificação da OS. A criação e os registros relacionados são persistidos em transação.

Consulta de status

Administradores consultam a situação por API autenticada com JWT. O cliente utiliza documento e token de acompanhamento para obter uma resposta restrita.

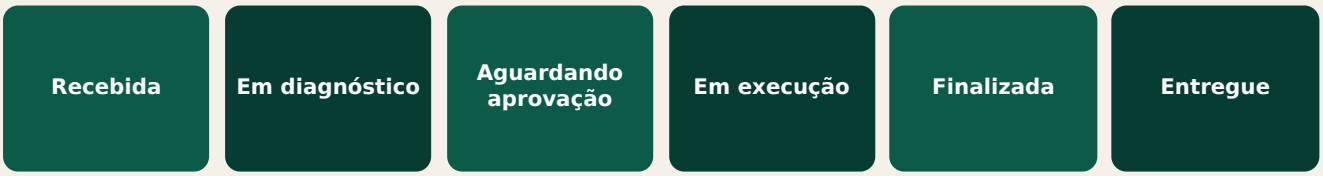
Orçamento e estoque

O webhook assinado por HMAC aceita aprovação e recusa. A aprovação inicia a execução e realiza baixa única do estoque; a recusa cancela a OS sem baixa.

Notificações e fila

Transições persistidas acionam e-mail e SMS conforme os contatos disponíveis. A fila prioriza execução, aprovação, diagnóstico e recebimento; em cada estado, a OS mais antiga vem primeiro.

Fluxo da Ordem de Serviço



A recusa do orçamento ou o cancelamento da oficina encerram a ordem no estado cancelada. Estados encerrados permanecem disponíveis para consulta histórica.

ARQUITETURA

02

Componentes e implantação

A aplicação mantém as regras de negócio no Domínio. A camada de Aplicação coordena os casos de uso; as interfaces HTTP e os adaptadores de infraestrutura conectam a API aos serviços externos.

Interface HTTP

Rotas, autenticação, validação de entrada e respostas REST.

Aplicação

Casos de uso, resultados e contratos de persistência e integração.

Domínio

Entidades, valores, orçamento e transições da Ordem de Serviço.

Interface HTTP → Aplicação → Domínio

Infraestrutura

Implementações dos contratos: Eloquent e PostgreSQL, JWT, e-mail por Mailpit ou SMTP e SMS por log local ou Twilio configurado.

Desenvolvimento local

Docker Compose executa API, PostgreSQL persistente e Mailpit. O mesmo Dockerfile é utilizado no build da imagem para Kind e CI.

Execução em Kubernetes

Terraform provisiona Kind e Metrics Server. A API usa Deployment e HPA por CPU e memória. PostgreSQL usa StatefulSet e volume persistente.

O cluster local permanece disponível para desenvolvimento. A entrega contínua cria um cluster temporário no runner do GitHub Actions e o destrói ao final da execução.

QUALIDADE COMPROVADA

03

Testes, APIs e escalabilidade

Em um clone público do repositório, a aplicação foi construída com Docker e testada com PostgreSQL. A cobertura informada corresponde às classes críticas configuradas para medição.

84
testes de domínio
152 asserções

192
testes de integração
693 asserções

100%
das linhas monitoradas
nas duas suítes

- ✓ Build Docker e Compose aprovados
- ✓ 13 migrations aplicadas
- ✓ Swagger e OpenAPI acessíveis
- ✓ 62 requisições HTTP validadas
- ✓ Webhook e estoque verificados
- ✓ Quatro cenários de contato testados
- ✓ CI/CD executada em runner hospedado
- ✓ Imagem publicada no GHCR

Escalabilidade e persistência

Uma carga de 120 segundos gerou 4.674 respostas HTTP 200, sem falhas. O HPA elevou a API de uma para quatro réplicas prontas, com métricas de CPU e memória disponíveis. Em verificação separada, o banco preservou dados após a recriação do pod com o mesmo PVC.

O teste de envio SMTP pelo Mailpit passou separadamente. Na validação HTTP, foram recebidos oito e-mails no Mailpit e registrados oito SMS simulados, sem contato com serviço pago.

SEGURANÇA E REFERÊNCIAS

04

Verificações e documentação técnica

O código e as dependências foram avaliados com Semgrep 1.89.0 e Composer Audit. As credenciais de execução são fornecidas por configuração privada, fora do repositório.

0 achados

O Semgrep executou 114 regras em 287 arquivos rastreados. O Composer Audit não reportou avisos de vulnerabilidade nas dependências verificadas.

Proteção dos contratos

JWT protege os endpoints administrativos; o webhook valida assinatura HMAC sobre o corpo da requisição. Testes verificaram assinaturas ausentes e inválidas.

Limites da análise

Oito arquivos tiveram análise parcial e duas regras excederam o tempo no PDF histórico. A ausência de achados não garante ausência de falhas.

Referências técnicas

- Repositório do projeto
- README: execução, arquitetura e infraestrutura
- Especificação OpenAPI 3.1
- Relatório técnico de vulnerabilidades
- Execução da integração e entrega contínuas